

TRIE DATA STRUCTURE

Efficient for the following operations on words in a dictionary.

- Search
- Insert
- delete
- prefix search
- Lexicographic ordering of words.

Example:

Insert("geek"), insert("geeks"), insert("bad"), insert("bat")

Search("bad") $\Rightarrow$ YES

Search("geek") $\Rightarrow$ NO

delete("geek") $\Rightarrow$ NO

Search("geek") $\Rightarrow$ NO

print() $\Rightarrow$ // bad bat geeks

Comparison with Hashing

	Time	Hashing
Search	$\theta(\text{word_len})$ in worst case	$\theta(\text{word_len})$ in average
Insert	"	"
Delete	"	"
Prefix Search	$\theta(\text{pref_len} + \text{output_len})$	Not supported
Lexicographic ordering	$\theta(\text{output_len})$	Not supported

Tree representation

```
struct TrieNode
```

```
{ TrieNode * child[26];
```

```
bool isEnd;
```

```
TrieNode()
```

```
{ isEnd = false;
```

```
for (int i=0; i<26; i++)
```

```
{ child[i] = NULL;
```

a 0

b 1

c 2

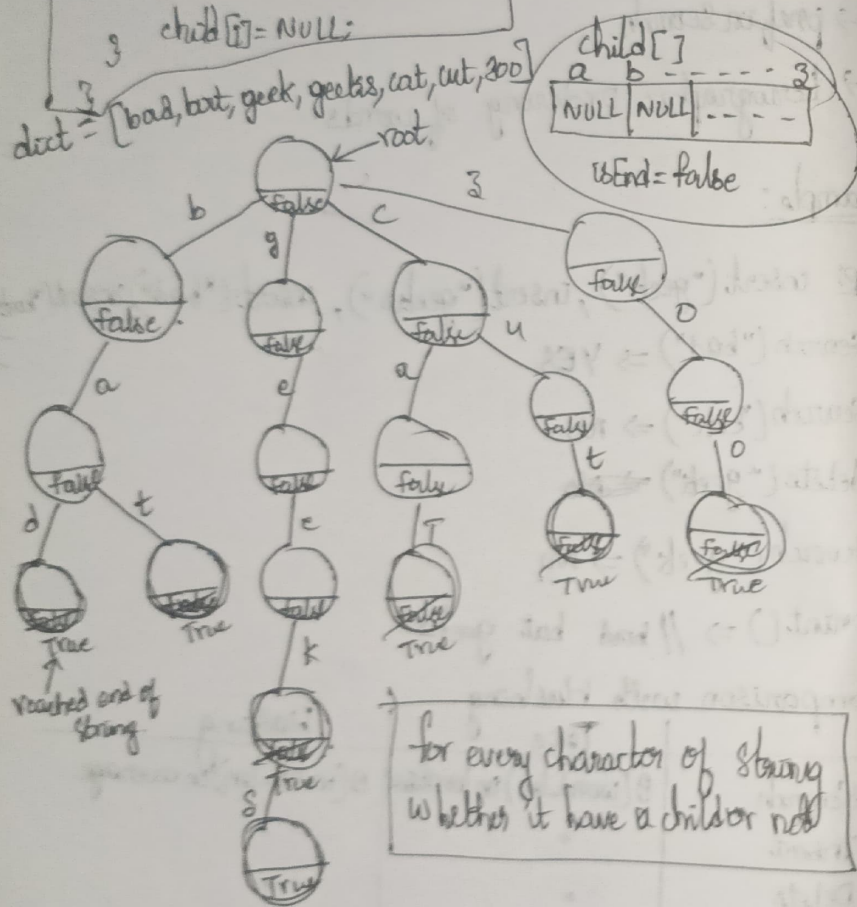
d 3

...

z 25

...

25 25



for every character of string
whether it have a child or not

Search:

bad => True

geek => True

gee => False

cat => False

[for every node we check do we have a child
Corresponding to the current character of the
string] => check isEnd of word true or not.

Search:

```
TrieNode * root;
```

```
bool search(string skey)
```

```
{ TrieNode * curr = root;
```

```
for (int i=0; i<skey.length(); i++)
```

```
{ int index = skey[i] - 'a';
```

```
if (curr->child[index] == NULL)
```

```
return false;
```

```
curr = curr->child[index];
```

```
return curr->isEnd;
```

```
}
```

string length
T.C = $O(n)$

Insert:

T.C = $O(n)$

```

TrieNode *root;
Void insert (String key)
{
    TrieNode *curr = root;
    for (int i=0; i < key.length(); i++)
    {
        int index = key[i] - 'a';
        if (curr->child[index] == NULL)
            curr->child[index] = new TrieNode();
        curr = curr->child[index];
    }
    curr->isEnd = true;
}

```

Trie Delete

Example cases in delete:

300: chain of 3 nodes (for '3', '0' and '0')
needs to be deleted.

geek: we only need to change isEnd as False.

bad: we need to delete one node from 'd'.

// i is initially '0'

TrieNode *delKey (TrieNode *root, string &key, int i)

```

{
    if (root == NULL) return NULL;
    if (i == key.length())

```

```

{
    root->isEnd = false;

```

```

    if (isEmpty (root) == true)

```

```

    {
        delete (root); root = NULL;
    }
    return root;
}

```

```

int index = key[i] - 'a';

```

```

root->child[index] = delKey (root->child[index], key, i+1);

```

```

if (isEmpty (root) && root->isEnd == false)

```

```

{
    delete (root); root = NULL;
}

```

```

return root;
}

```

```

bool isEmpty (TrieNode *root)
{

```

```

    for (int i=0; i < 26; i++)

```

```

    {
        if (root->child[i] != NULL)

```

```

            return false;
    }

```

```

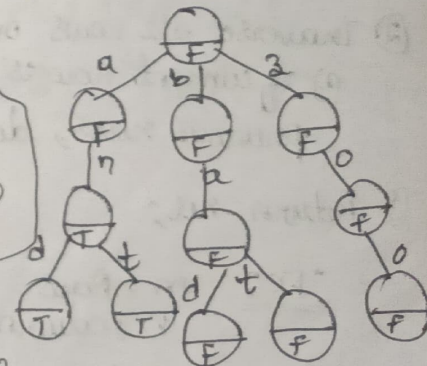
    return true;
}

```

T.C = $O(n)$

key length

[an, and, ant, bad, bat, 300]



Count Distinct Rows in a Binary Matrix

IP: $\text{mult}[i][j] = \begin{bmatrix} 1, 0, 0 \\ 1, 1, 1 \\ 1, 0, 0 \\ 1, 1, 1 \end{bmatrix}$

O/p: 2

$$\exists p: \text{mat}[\][\] = \begin{bmatrix} 1, 1, 0, 0 \\ 1, 1, 0, 0 \\ 1, 1, 0, 0 \\ 1, 1, 0, 0 \end{bmatrix}$$

O/p: 1

Nine

- ① Initialise : $res = 0$;
- ② Traverse all rows one by one.
a) If current row is not same as any of the previous rows, do res++
- ③ Return res ;

T.C : m : Rows
n : columns

$$O(m \times m \times n)$$
 $O(m^2n)$

```
bool mat[M][N];
struct TrieNode {
    TrieNode* child[2];
    TrieNode()
    {
        child[0] = NULL;
        child[1] = NULL;
    }
};
```

3;³

```
bool insert(Treenode *root, int row)
```

```
2- TrueNode *curr = next;  
   bool flag = false;
```

```
for(int i=0; i<N; i++)
```

```
int endera = mat[nrow][i];
```

if (curr->child[indera] == NULL)

```
curr->child[index] = new TrieNode();
```

flag = true;

```
curr = curr->child[indx];
```

return flag;

```
int CountDist()
```

```
TreeNode* root = new TreeNode();
```

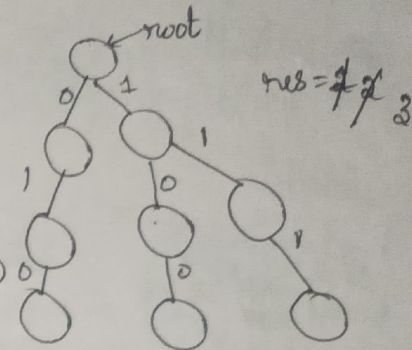
```
int res=0;
```

```
for (int now = 0; now < M; now++)
```

if (insert(grid, row) == true)
 succ;

Return red;

3

$$\begin{bmatrix} [1, 0, 0], \\ [1, 1, 1], \\ [1, 0, 0], \\ [0, 1, 0] \end{bmatrix}$$


$$T.C = O(mn)$$

$$A.S = O(mn)$$